Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ
Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЁТ

о выполнении лабораторной работы №2
по дисциплине «Проектирование баз знаний»

на тему:

Проектирование и разработка информационной системы «Банк данных технологий создания продуктов»

(Вариант №23)

Студент гр. 421702
А. В. Сайковский

Преподаватель
Н. А. Гуменный

Минск 2026

Содержание

1 ПОСТАНОВКА ЗАДАЧИ 3
1.1 Описание предметной области 3
1.2 Функциональные требования 3
1.3 Нефункциональные требования 4
2 КОНЦЕПТУАЛЬНОЕ ПРОЕКТИРОВАНИЕ 5
2.1 Спецификация ВИ-01 «Управлять записями» 5
2.2 Спецификация ВИ-02 «Получить прайс-лист поставщика» . . 6
2.3 Спецификация ВИ-03 «Найти товары с минимальной калорийностью» 7
2.4 Спецификация ВИ-04 «Получить товары с рецептами» 7
3 ЛОГИЧЕСКОЕ ПРОЕКТИРОВАНИЕ 9
3.1 Описание сущностей 10
3.2 Связи и правила удаления 10
4 ФИЗИЧЕСКОЕ ПРОЕКТИРОВАНИЕ 12
4.1 Ограничения и индексы 13
5 ОСНОВНЫЕ ЗАПРОСЫ, ФУНКЦИИ, ПРОЦЕДУРЫ И ТРИГГЕРЫ 14
5.1 Общий CRUD-запрос чтения 14
5.2 Создание записи 14
5.3 Чтение товара вместе с группой 15
5.4 Прайс-лист поставщика 15
5.5 Расчёт минимальной калорийности 15
5.6 Функции серверного приложения 16
5.7 Generic-репозиторий 17
5.8 Хранимые процедуры и триггеры 18
6 ИНТЕРФЕЙС ПРИЛОЖЕНИЯ 19
6.1 Главное окно 19
6.2 Окно создания записи 19
6.3 Окно изменения записи 20
6.4 Интерактивная документация API 21
6.5 Обмен данными с сервером 22
7 ИНСТРУМЕНТАЛЬНЫЕ СРЕДСТВА 23
7.1 Конфигурация и запуск 23

7.2 Тестирование 24
Заключение **25**
Список использованных источников **26**

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Описание предметной области

Предметной областью разрабатываемой информационной системы является учёт технологий производства пищевых продуктов. Производственное предприятие выпускает товары разных групп: хлебобулочные, кондитерские и молочные изделия, напитки, соусы и другие виды продукции. Для каждого товара может существовать один или несколько рецептов. Рецепт имеет уникальный номер, название, текстовое описание и автора.

Состав рецепта определяется перечнем ингредиентов. Для каждого ингредиента в конкретном рецепте необходимо знать его массу в граммах на один килограмм готового продукта и способ подготовки: просеивание, нагревание, смешивание, взбивание и т. д. Один ингредиент может входить во многие рецепты, а один рецепт состоит из многих ингредиентов. Калорийность готового продукта рассчитывается на основании калорийности и массы составляющих.

Ингредиенты приобретаются у поставщиков. Факт поставки оформляется накладной, которая содержит поставщика, номер и дату. Каждая позиция накладной указывает ингредиент, поставленное количество и цену единицы. Один поставщик может оформить множество накладных, а одна накладная содержит множество позиций.

Система должна обеспечивать централизованное хранение данных, соблюдение ссылочной целостности, просмотр и изменение записей через web-интерфейс, а также формирование аналитических отчётов.

1.2 Функциональные требования

Пользователь должен иметь возможность:

1. просматривать, добавлять, изменять и удалять товары;
2. вести справочники групп товаров и способов подготовки;
3. вести сведения об ингредиентах и их калорийности;
4. вести сведения об авторах и рецептах;
5. вести сведения о поставщиках и накладных;

6. получать прайс-лист поставщика на выбранную дату;
7. определять товары с минимальной калорийностью;
8. получать список товаров и связанных с ними рецептов.

1.3 Нефункциональные требования

Приложение должно работать асинхронно, проверять входные данные, использовать PostgreSQL и ручные параметризованные SQL-запросы. Значения, введённые во frontend, не должны объединяться со строкой SQL. Ошибки целостности должны приводить к откату транзакции и понятному HTTP-ответу. База данных должна запускаться воспроизводимо через Docker Compose, а основные операции должны быть покрыты автоматическими тестами.

2 КОНЦЕПТУАЛЬНОЕ ПРОЕКТИРОВАНИЕ

Для описания поведения системы выбрана диаграмма вариантов использования (Use Case). Основным актором является пользователь приложения — сотрудник, который ведёт технологические и закупочные данные. Диаграмма представлена на рисунке 1.

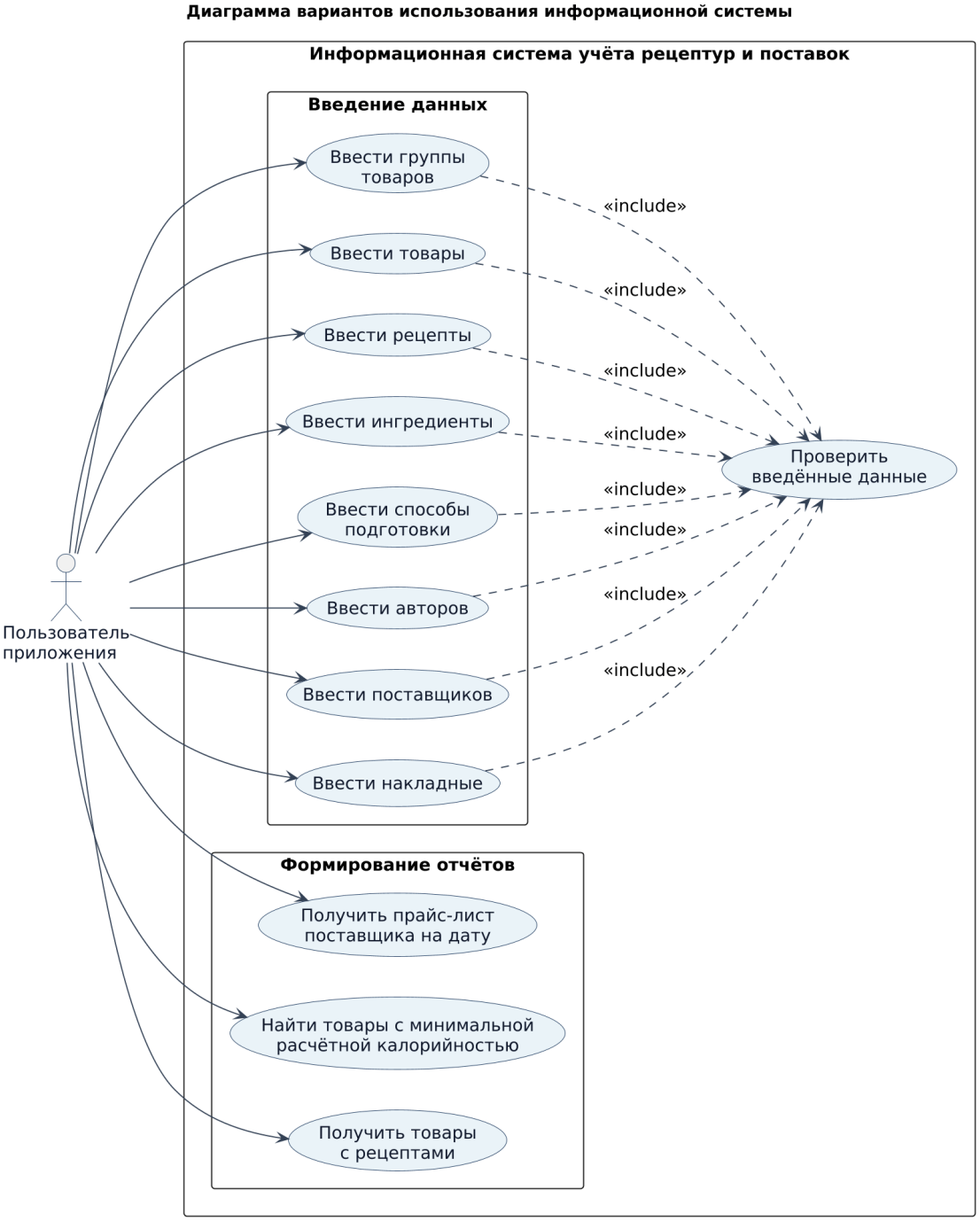


Рис. 1: Диаграмма вариантов использования системы

2.1 Спецификация ВИ-01 «Управлять записями»

Поле	Описание
Идентификатор	ВИ-01.
Цель	Создать, просмотреть, изменить или удалить запись выбранного ресурса.
Актор	Пользователь.
Предусловия	Приложение запущено; соединение с базой данных установлено.
Основной сценарий	1) Пользователь открывает раздел. 2) Система загружает список. 3) Пользователь выбирает действие и вводит данные. 4) Система проверяет данные. 5) Выполняется параметризованный SQL-запрос. 6) Транзакция фиксируется. 7) Обновлённый список выводится на экран.
Альтернативы	Неверные данные — HTTP 422; запись не найдена — HTTP 404; нарушено ограничение БД — HTTP 409 с откатом транзакции.
Постусловия	База данных изменена только при успешном завершении операции.

2.2 Спецификация ВИ-02 «Получить прайс-лист поставщика»

Поле	Описание
Идентификатор	ВИ-02.
Цель	Получить цены ингредиентов определённого поставщика на выбранную дату.
Актор	Пользователь.
Предусловия	Поставщик зарегистрирован в системе.
Основной сценарий	1) Пользователь задаёт поставщика и дату. 2) Система проверяет существование поставщика. 3) Выполняется соединение накладных, позиций и ингредиентов. 4) Возвращаются наименования, цены и номера накладных.

Поле	Описание
Альтернативы	Поставщик отсутствует — HTTP 404; поставки на дату отсутствуют — возвращается пустой список позиций.
Постусловия	Данные не изменяются.

2.3 Спецификация ВИ-03 «Найти товары с минимальной калорийностью»

Поле	Описание
Идентификатор	ВИ-03.
Цель	Определить рецепты с наименьшей расчётной калорийностью.
Актор	Пользователь.
Предусловия	В БД присутствуют рецепты и их компоненты.
Основной сценарий	1) Пользователь запускает отчёт. 2) Для каждого рецепта суммируются произведения массы и калорийности ингредиентов. 3) Определяется минимум. 4) Система возвращает все рецепты с минимальным значением.
Альтернативы	Рецепты с компонентами отсутствуют — минимум равен нулю, список пуст.
Постусловия	Данные не изменяются.

2.4 Спецификация ВИ-04 «Получить товары с рецептами»

Поле	Описание
Идентификатор	ВИ-04.
Цель	Получить иерархический перечень товаров и вариантов их рецептов.
Актор	Пользователь.
Предусловия	В системе зарегистрированы товары и рецепты.

Поле	Описание
Основной сценарий	1) Пользователь открывает отчёт. 2) Система соединяет товары, группы и рецепты. 3) Строки группируются по товару. 4) Возвращается JSON со списком рецептов внутри каждого товара.
Альтернативы	Подходящие записи отсутствуют — возвращается пустой список.
Постусловия	Данные не изменяются.

3 ЛОГИЧЕСКОЕ ПРОЕКТИРОВАНИЕ

Логическая модель содержит основные сущности предметной области, их атрибуты и мощности связей, но не зависит от конкретных типов PostgreSQL. ER-диаграмма приведена на рисунке 2.

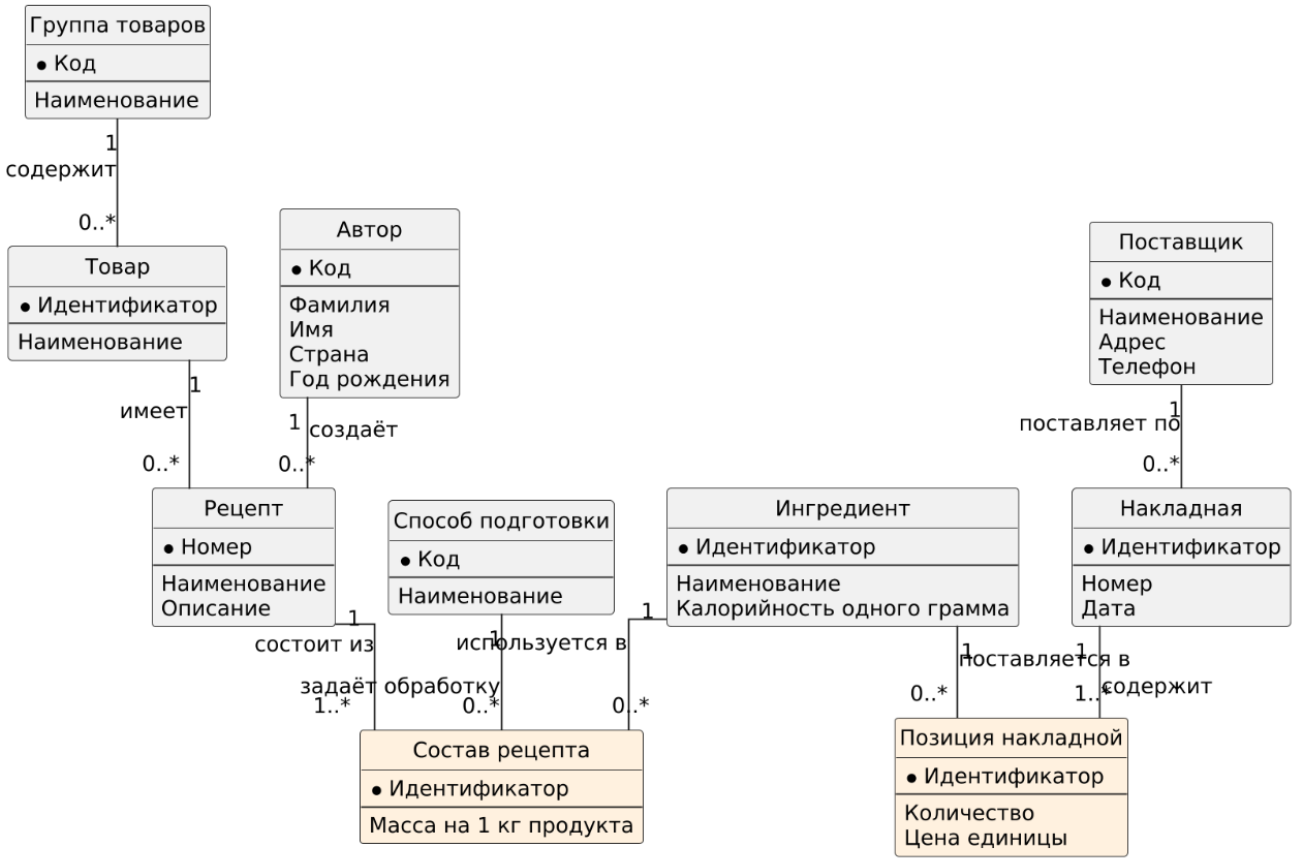


Рис. 2: Логическая ER-диаграмма предметной области

3.1 Описание сущностей

Сущность	Назначение и основные атрибуты
Группа товаров	Код и наименование категории продукции.
Товар	Наименование производимого продукта и ссылка на группу.
Автор	Код, фамилия, имя, страна и год рождения автора рецепта.
Рецепт	Уникальный номер, название, описание, товар и автор.
Ингредиент	Уникальное наименование и калорийность одного грамма.
Способ подготовки	Код и наименование технологической операции.
Состав рецепта	Связь рецепта, ингредиента и способа подготовки; количество граммов на килограмм.
Поставщик	Код, наименование, адрес и телефон.
Накладная	Поставщик, номер документа и дата поставки.
Позиция накладной	Ингредиент, количество и цена единицы.

3.2 Связи и правила удаления

Родитель	Мощность	Дочерняя сущность	Правило
Группа товаров	1:N	Товар	RESTRICT
Товар	1:N	Рецепт	CASCADE
Автор	1:N	Рецепт	RESTRICT
Рецепт	1:N	Состав рецепта	CASCADE
Ингредиент	1:N	Состав рецепта	RESTRICT
Способ подготовки	1:N	Состав рецепта	RESTRICT
Поставщик	1:N	Накладная	RESTRICT

Родитель	Мощность	Дочерняя сущность	Правило
Накладная	1:N	Позиция накладной	CASCADE
Ингредиент	1:N	Позиция накладной	RESTRICT

`CASCADE` используется для зависимых записей, которые не имеют смысла без владельца. `RESTRICT` запрещает удаление справочной записи, если на неё уже ссылаются бизнес-данные.

4 ФИЗИЧЕСКОЕ ПРОЕКТИРОВАНИЕ

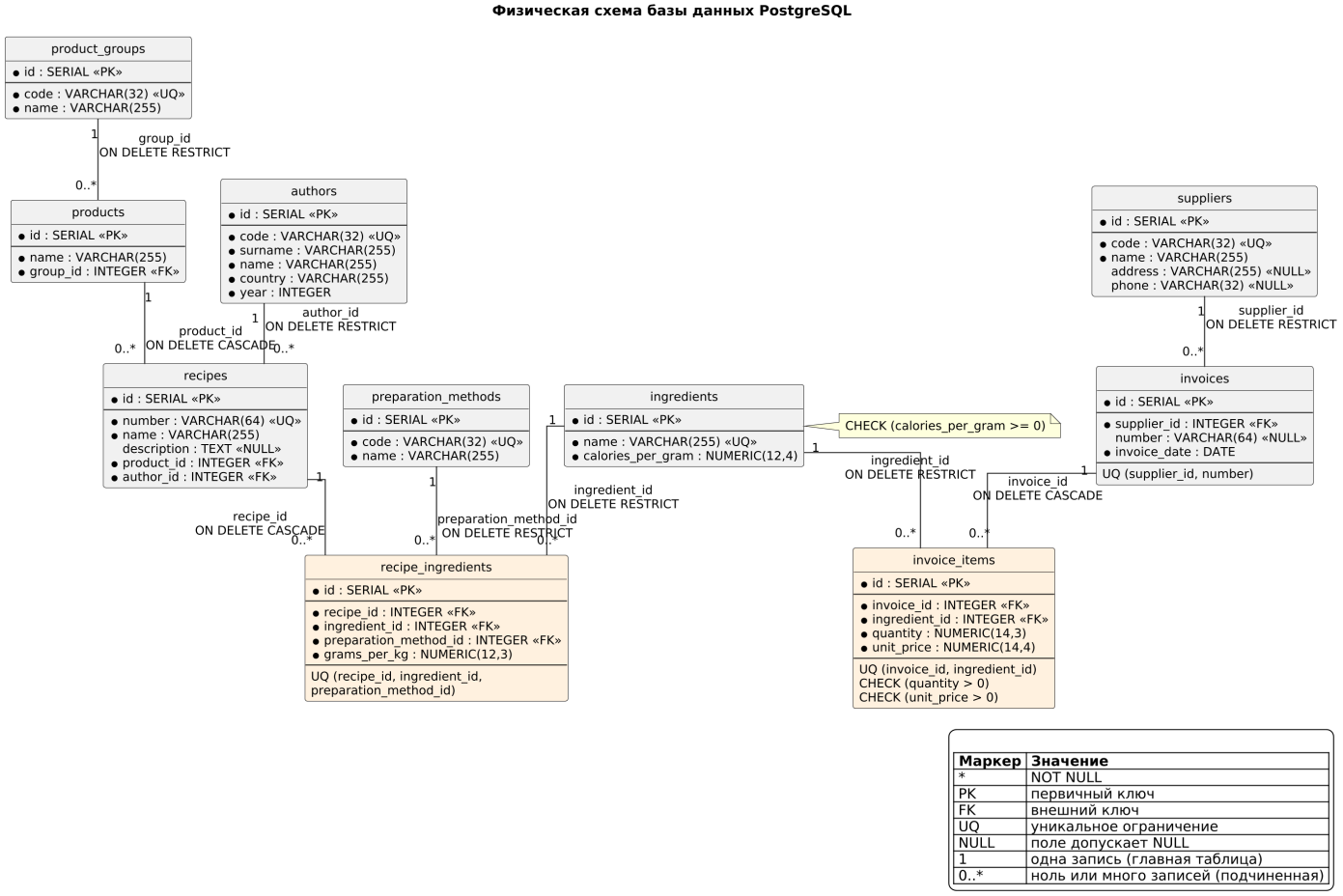


Рис. 3: Физическая схема базы данных PostgreSQL

Физическая модель ориентирована на PostgreSQL. Все таблицы имеют surrogate primary key `id` типа `SERIAL`. Денежные, количественные и калорийные значения хранятся в `NUMERIC`, чтобы исключить ошибки двоичной арифметики с плавающей точкой. Схема представлена на рисунке 3.

4.1 Ограничения и индексы

На внешние ключи и дату накладной созданы индексы. Уникальные ограничения защищают коды справочников, номер рецепта, пару «поставщик–номер накладной» и повтор ингредиента внутри одной накладной. Ограничения `CHECK` запрещают отрицательную калорийность, неположительные количество и цену.

Фрагмент физической схемы:

```sql
-- Позиция накладной не может существовать без накладной.
CREATE TABLE invoice_items (
    id            SERIAL PRIMARY KEY,
    invoice_id    INTEGER NOT NULL
                  REFERENCES invoices(id) ON DELETE CASCADE,
    ingredient_id INTEGER NOT NULL
                  REFERENCES ingredients(id) ON DELETE
                   ↪  RESTRICT,
    quantity      NUMERIC(14, 3) NOT NULL,
    unit_price    NUMERIC(14, 4) NOT NULL,

    -- Один ингредиент встречается в накладной не более одного
     ↪  раза.
    CONSTRAINT uq_invoice_items_invoice_id_ingredient_id
        UNIQUE (invoice_id, ingredient_id),

    -- Некорректные количество и цена отвергаются самой СУБД.
    CONSTRAINT ck_invoice_items_quantity_positive CHECK
     ↪  (quantity > 0),
    CONSTRAINT ck_invoice_items_unit_price_positive CHECK
     ↪  (unit_price > 0)
);
```

Начальное наполнение находится в `sql/02_seed.sql`. Оно идемпотентно: повторный запуск использует `ON CONFLICT DO NOTHING` и `NOT EXISTS`, поэтому не создаёт дубликаты.

5 ОСНОВНЫЕ ЗАПРОСЫ, ФУНКЦИИ, ПРОЦЕДУРЫ И ТРИГГЕРЫ

5.1 Общий CRUD-запрос чтения

SQLAlchemy используется для соединения и выполнения команд, но не для генерации ORM-запросов. Значение `id` передаётся отдельно от SQL-текста, что защищает запрос от SQL-инъекций.

```python
async def get(self, id: int) -> T | None:
    # Имена таблицы и столбцов заданы в серверном коде
    ↪ репозитория.
    # Пользователь не может передать их через HTTP.
    sql = text(
        f"SELECT id, {', '.join(self._columns)} "
        f"FROM {self._table} WHERE id = :id"
    )

    # Значение id отправляется драйверу как bind-параметр.
    row = (
        await self._session.execute(sql, {"id": id})
    ).mappings().one_or_none()

    return self._from_row(row) if row is not None else None
```

5.2 Создание записи

Оператор `RETURNING` возвращает созданную строку и назначенный PostgreSQL идентификатор без дополнительного `SELECT`.

```sql
-- name и group_id поступают из проверенной Pydantic-схемы,
-- но передаются в запрос только как параметры.
INSERT INTO products (name, group_id)
VALUES (:name, :group_id)
RETURNING id, name, group_id;
```

После успешного выполнения репозиторий вызывает `commit`. При `IntegrityError` выполняется `rollback`, а клиент получает HTTP 409.

5.3 Чтение товара вместе с группой

Запрос с `JOIN` используется вместо ORM-загрузки отношения. Псевдонимы позволяют отличить поля товара от полей группы.

```sql
-- Возвращает товар и данные его группы одним запросом.
SELECT
    p.id,
    p.name,
    p.group_id,
    pg.id   AS group_object_id,
    pg.code AS group_code,
    pg.name AS group_name
FROM products AS p
JOIN product_groups AS pg ON pg.id = p.group_id
WHERE p.id = :id;
```

5.4 Прайс-лист поставщика

Запрос выбирает цены только из накладных заданного поставщика за указанную дату. Оба значения являются bind-параметрами.

```sql
-- Прайс-лист содержит ингредиент, цену и исходный номер
↪  накладной.
SELECT
    ing.id   AS ingredient_id,
    ing.name AS ingredient_name,
    ii.unit_price,
    inv.number AS invoice_number
FROM invoices AS inv
JOIN invoice_items AS ii ON ii.invoice_id = inv.id
JOIN ingredients AS ing ON ing.id = ii.ingredient_id
WHERE inv.supplier_id = :supplier_id
  AND inv.invoice_date = :price_date
ORDER BY ing.id, inv.id;
```

5.5 Расчёт минимальной калорийности

Для каждого рецепта вычисляется калорийность одного килограмма:

$$K_{1\,\text{кг}} = \sum_{i=1}^{n} k_i \cdot m_i, \qquad K_{100\,\text{г}} = \frac{K_{1\,\text{кг}}}{10}.$$

CTE сначала агрегирует калорийность каждого рецепта, затем внешний запрос выбирает все строки с минимальным значением.

```sql
-- Расчёт выполняется средствами PostgreSQL, а не циклом Python.
WITH recipe_calories AS (
    SELECT
        p.id AS product_id,
        p.name AS product_name,
        pg.name AS group_name,
        r.id AS recipe_id,
        r.name AS recipe_name,
        SUM(ing.calories_per_gram * ri.grams_per_kg)
            AS calories_per_1kg
    FROM recipes AS r
    JOIN products AS p ON p.id = r.product_id
    JOIN product_groups AS pg ON pg.id = p.group_id
    JOIN recipe_ingredients AS ri ON ri.recipe_id = r.id
    JOIN ingredients AS ing ON ing.id = ri.ingredient_id
    GROUP BY p.id, p.name, pg.name, r.id, r.name
)
SELECT *
FROM recipe_calories
WHERE calories_per_1kg = (
    SELECT MIN(calories_per_1kg)
    FROM recipe_calories
)
ORDER BY recipe_id;
```

5.6 Функции серверного приложения

Прикладные функции реализованы асинхронными методами Python. Например, `get_session` управляет транзакцией на протяжении HTTP-запроса:

```python
async def get_session() -> AsyncGenerator[AsyncSession, None]:
    async with async_session_maker() as session:
```

```
        try:
            # Сессия передаётся endpoint через Depends.
            yield session
            await session.commit()
        except IntegrityError as exc:
            # При нарушении ограничений изменения отменяются.
            await session.rollback()
            raise exc
        except Exception:
            # Любая другая ошибка также не оставляет частичных
             ↪  изменений.
            await session.rollback()
            raise
```

Функция dependency injection создаёт сервис с той же сессией:

```
def get_product_service(
    session: Annotated[AsyncSession, Depends(get_session)],
) -> ProductService:
    # Endpoint получает готовый сервис и не управляет БД
     ↪  напрямую.
    return ProductService(session)
```

5.7 Generic-репозиторий

Общий интерфейс параметризован типом сущности T. Поэтому один CRUD алгоритм применяется к товарам, авторам, поставщикам и другим сущностям без потери информации о возвращаемом типе.

```
T = TypeVar("T", bound=Base)

class IRepository(ABC, Generic[T]):
    @abstractmethod
    async def get(self, id: int) -> T | None:
        raise NotImplementedError

    @abstractmethod
    async def list(
        self, *, offset: int = 0, limit: int = 100
```

```
    ) -> list[T]:
        raise NotImplementedError

class BaseRepository(IRepository[T], Generic[T]):
    # Реализует ручные SELECT, INSERT, UPDATE и DELETE.
    ...
```

5.8 Хранимые процедуры и триггеры

В текущей версии базы данных пользовательские хранимые процедуры и триггеры не применяются. Это осознанное архитектурное решение: CRUD и отчётная логика сосредоточены в сервисах и репозиториях FastAPI, а PostgreSQL отвечает за хранение, связи, проверки и агрегирование.

Функции, которые часто возлагаются на триггеры, реализованы декларативными средствами БД:

- каскадное удаление — внешними ключами `ON DELETE CASCADE`;
- запрет удаления используемых справочников — `ON DELETE RESTRICT`;
- проверка положительных значений — ограничениями `CHECK`;
- предотвращение дублей — ограничениями `UNIQUE`;
- атомарность — транзакциями `commit`/`rollback`.

Следовательно, текстов пользовательских `CREATE PROCEDURE`, `CREATE FUNCTION` и `CREATE TRIGGER` в проекте нет. Такой подход не скрывает бизнес-логику внутри СУБД и делает её доступной для тестирования в Python.

6 ИНТЕРФЕЙС ПРИЛОЖЕНИЯ

Frontend реализован в файле `static/index.html`. Сервер FastAPI отдаёт его по корневому адресу `/`. Страница использует HTML, CSS и JavaScript без отдельного frontend-фреймворка. Все данные загружаются через REST API с префиксом `/api/v1`.

6.1 Главное окно

Главное окно содержит навигацию и таблицы основных сущностей: товаров, рецептов, поставщиков, авторов, ингредиентов, групп товаров, способов подготовки и накладных. После события `DOMContentLoaded` функция `loadAllTables` вызывает API для каждого раздела. Полученный JSON передаётся в универсальную функцию построения таблицы.

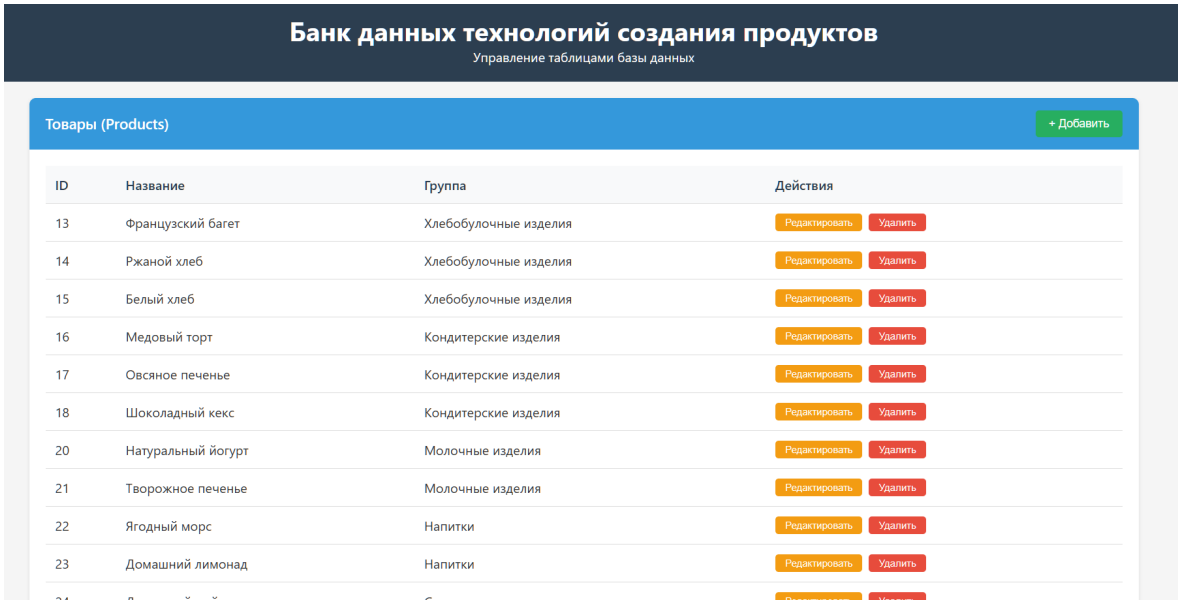


Рис. 4: Главное окно приложения

6.2 Окно создания записи

Кнопка «Добавить» открывает модальное окно. Состав формы определяется объектом `tableConfigs`: для каждого ресурса в нём заданы поля, подписи, типы и признак обязательности. После подтверждения JavaScript отправляет `POST`-запрос с JSON-телом. При успехе окно закрывается, а таблица перезагружается.

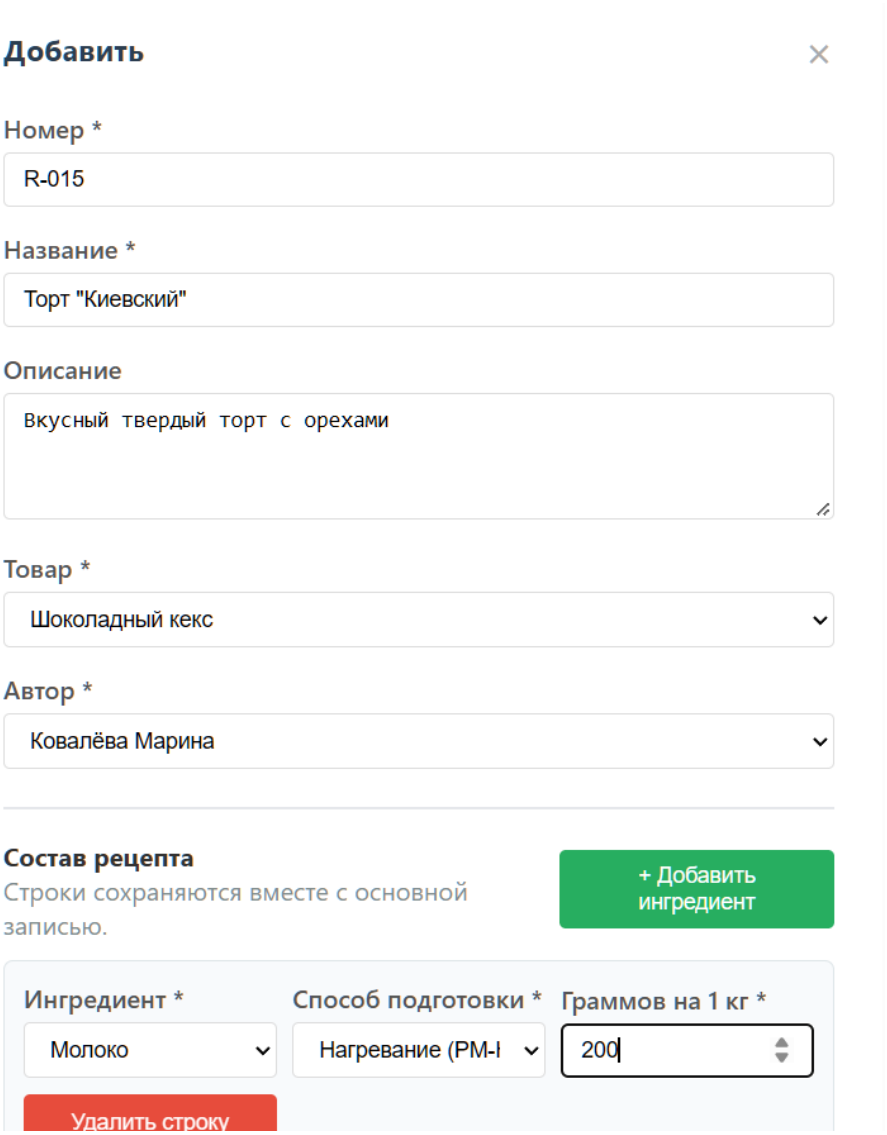


Рис. 5: Окно создания записи

6.3 Окно изменения записи

Перед редактированием frontend получает актуальную запись запросом `GET /<resource>/<id>` и заполняет форму. Изменённые значения передаются методом `PUT`. Не переданные необязательные поля сохраняют прежние значения благодаря логике сервисного слоя.

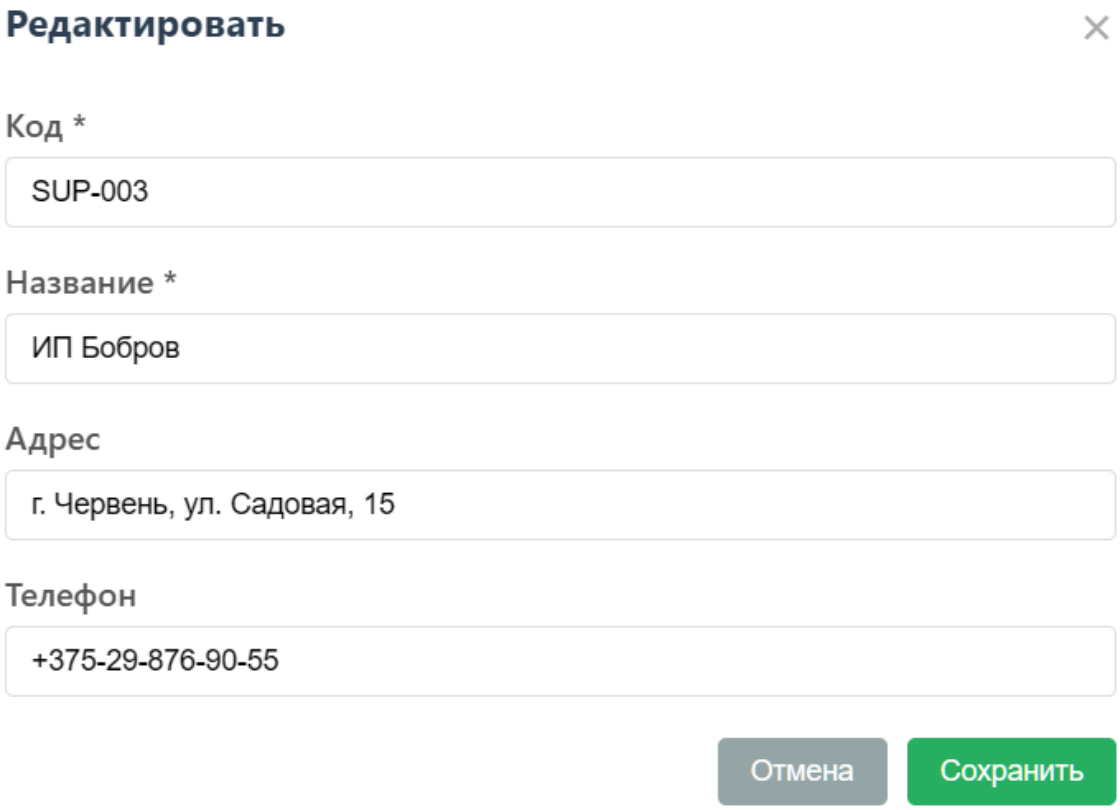


Рис. 6: Окно изменения записи

6.4 Интерактивная документация API

FastAPI автоматически формирует Swagger UI по адресу `/docs`. В нём можно просмотреть схемы Pydantic, параметры запросов, коды ответов и выполнить любой endpoint без стороннего HTTP-клиента. Альтернативная документация ReDoc доступна по адресу `/redoc`.

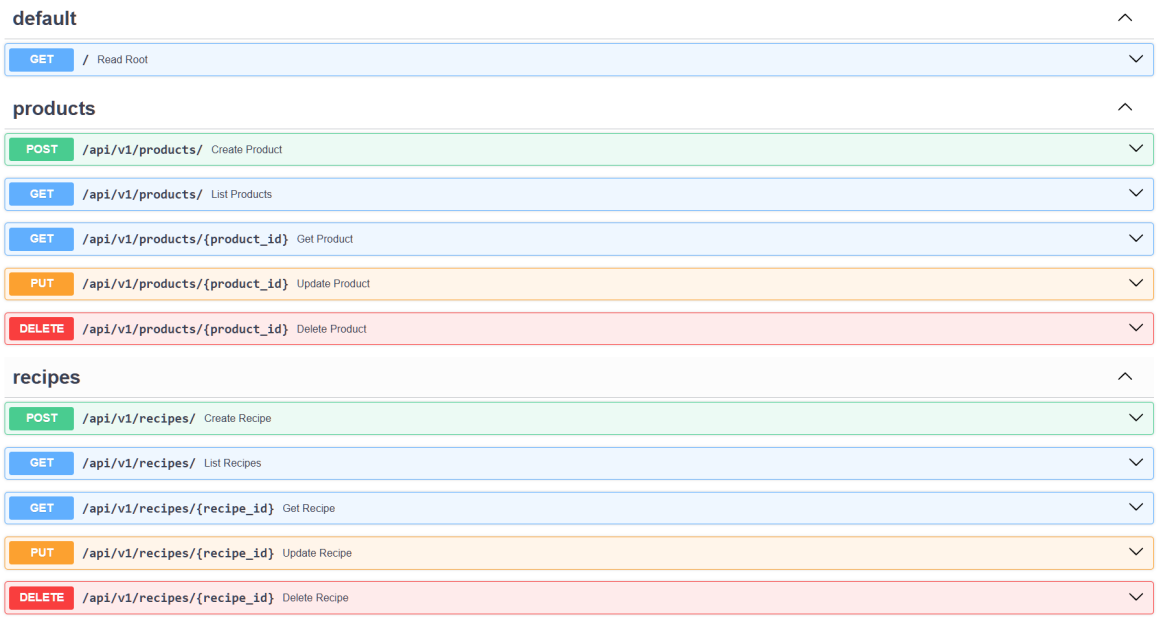


Рис. 7: Интерактивная документация Swagger UI

6.5 Обмен данными с сервером

Универсальный вызов списка имеет следующий вид:

```
// apiPath берётся из конфигурации выбранной таблицы.
const response = await fetch(
    `${API_BASE}${config.apiPath}/`
);

if (!response.ok) {
    throw new Error('Failed to fetch');
}

// Сервер возвращает JSON, из которого строится HTML-таблица.
const data = await response.json();
renderTable(tableName, data, config);
```

7 ИНСТРУМЕНТАЛЬНЫЕ СРЕДСТВА

Средство	Назначение
Python 3.11+	Язык реализации серверной части и тестов.
FastAPI	Маршрутизация HTTP, dependency injection, обработка запросов и генерация OpenAPI.
Pydantic 2	Валидация JSON и сериализация ответов.
PostgreSQL 17	Реляционная СУБД, внешние ключи, ограничения, индексы, соединения и агрегирование.
SQLAlchemy 2 AsyncIO	Пул соединений, асинхронная сессия, транзакции и выполнение ручного SQL через `text`.
asyncpg	Асинхронный драйвер PostgreSQL.
HTML, CSS, JavaScript	Реализация web-интерфейса и обращение к API через `fetch`.
Docker Compose	Запуск PostgreSQL, постоянный volume и автоматическое подключение DDL/seed-скриптов.
pytest, pytest-asyncio	Модульные и интеграционные тесты асинхронного кода.
httpx	Проверка ASGI-приложения и PostgreSQL smoke-тест.
Ruff	Статический анализ и форматирование Python-кода.
Git	Контроль версий проекта.
MiKTeX, XeLaTeX	Подготовка и сборка отчёта.

7.1 Конфигурация и запуск

Параметры соединения находятся в `.env` и читаются через `pydantic-settings`. Пароль не зашит в исходный код. Docker Compose подключает `sql/01_schema.sql` и `sql/02_seed.sql` как инициализационные скрипты PostgreSQL.

```
docker compose up -d
python -m uvicorn app.main:app --reload
```

После запуска интерфейс доступен по адресу `http://localhost:8000/`, Swagger UI — по адресу `http://localhost:8000/docs`.

7.2 Тестирование

Автоматические тесты проверяют CRUD, параметризацию SQL, работу с `Decimal` и датами, загрузку связанных объектов и три отчётных запроса. Строка, внешне похожая на SQL-инъекцию, сохраняется как обычное значение, что подтверждает корректность bind-параметров.

Скрипт `scripts/smoke_postgres.py` дополнительно проверяет полный HTTP-цикл с реальной PostgreSQL: чтение коллекций, создание, получение, обновление и удаление товара, а также запуск всех отчётов.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы выполнено концептуальное, логическое и физическое проектирование информационной системы для учёта технологий создания продуктов. На концептуальном уровне определены актор, варианты использования и сценарии взаимодействия. Логическая модель описывает сущности и мощности связей, а физическая схема задаёт таблицы PostgreSQL, типы, ключи, индексы и ограничения целостности.

На основе спроектированной базы разработано асинхронное web-приложение. Frontend взаимодействует с REST API, сервер разделён на контроллеры, сервисы и репозитории. Все запросы написаны вручную и используют bind-параметры. Generic-репозиторий уменьшает дублирование CRUD-кода, а Pydantic и ограничения PostgreSQL обеспечивают многоуровневую проверку данных.

Работоспособность подтверждена модульными тестами и smoke-тестом на реальной PostgreSQL. Разработанная структура допускает добавление новых справочников, отчётов и интерфейсных представлений без изменения базовых принципов системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] PostgreSQL 17 Documentation. — https://www.postgresql.org/docs/17/.

[2] FastAPI Documentation. — https://fastapi.tiangolo.com/.

[3] SQLAlchemy 2.0 Documentation. — https://docs.sqlalchemy.org/en/20/.

[4] Pydantic Documentation. — https://docs.pydantic.dev/latest/.

[5] Docker Compose Documentation. — https://docs.docker.com/compose/.

[6] Object Management Group. Unified Modeling Language. — https://www.omg.org/spec/UML/.